

COMP4601 Project Demonstration Plan

FPGA-Accelerated Image Compression on AMD Kria KV260

Brief Objective

This demonstration shows the end-to-end operation of a hardware-accelerated grayscale image-compression pipeline on the AMD Kria KV260 FPGA. The project implements a JPEG-like compression pipeline consisting of:

PGM Image -> Level Shift -> DCT -> Quantisation -> Zigzag / Run-Length Encoding -> .gsc compressed file

The demonstration will show that:

- Grayscale .pgm images can be compressed correctly on the Kria KV260;
- The computationally intensive compression stages execute in FPGA programmable logic;
- Compressed file size is smaller than the original image size;
- FPGA kernel execution time can be measured separately from overall host execution time;
- The accelerated FPGA implementation provides measurable speedup over the software-only implementation.
- The decompressed output can be compared against the original image to quantify the loss introduced by compression using pixel-error statistics and PSNR.

Part 1: Software-Only Compression Baseline

Goal: Establish the CPU software baseline and record its profiling output before demonstrating FPGA acceleration.

Step 1 - Login to the board

After logging in to the Kria board, move to the software implementation directory:

```
cd software
```

Step 2 - Run software compression

```
./software_compress_images output_software
```

Show the terminal output while the software implementation compresses the input images.

Step 3 - Record software profiling output

Make a note of the software profiling values, especially:

- total processing time;
- average processing time per image
- time spent in major stages such as DCT, quantisation, zigzag and run-length encoding.

Part 2: Run the Optimised FPGA Implementation

Goal: Run the most optimised accelerated implementation and compare its measured kernel time against the software baseline.

Step 1 - Move to the FPGA implementation

```
cd ..  
cd compression_3
```

Step 2 - Run the accelerated compressor

```
./compression_host compression_3 images output_3
```

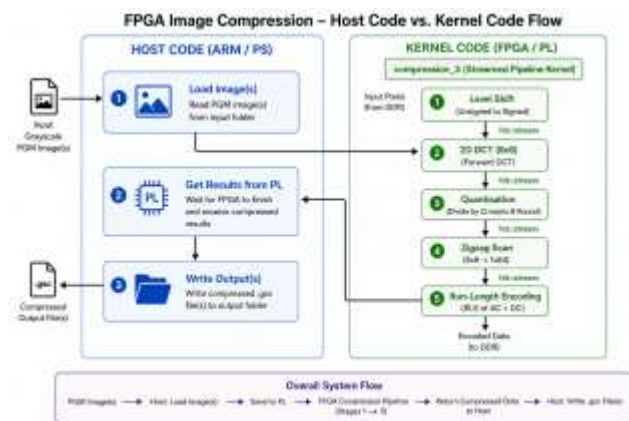
Show the per-image and batch profiling output from the host program. Highlight the FPGA kernel time separately from the overall application time.

Step 3 - Show the acceleration

Compare the FPGA kernel time with the CPU time previously spent in the stages that are now implemented inside the hardware kernel.

Part 3: Explain the Accelerated FPGA Pipeline

Goal: Explain what changed architecturally and why this FPGA version is faster than the presented in Final Presentation.



Timing Diagram for Image Compression Pipeline

Image	1	2	3	4	5	6	7	8	9	10	11	12
Image 0	LOAD	DCT	QUANT	ZIG-ZAG	RLE	WRITE						
Image 1		LOAD	DCT	QUANT	ZIG-ZAG	RLE	WRITE					
Image 2			LOAD	DCT	QUANT	ZIG-ZAG	RLE	WRITE				
Image 3				LOAD	DCT	QUANT	ZIG-ZAG	RLE	WRITE			
Image 4					LOAD	DCT	QUANT	ZIG-ZAG	RLE	WRITE		
Image 5						LOAD	DCT	QUANT	ZIG-ZAG	RLE	WRITE	
Image 6							LOAD	DCT	QUANT	ZIG-ZAG	RLE	WRITE

Key points to explain:

- The current implementation uses a single optimised compression kernel rather than different kernels for different stages.
- Streaming/dataflow allows different pipeline stages to work on different data at the same time.
- The DCT is a major compute-heavy stage, so hardware implementation and pipelining reduce its processing cost substantially.
- Kernel time measures how long the hardware compute section takes; total time is larger because it may include host (reading/writing images) and transfer overhead.

Part 4: Decompress the FPGA Output

Goal: Verify that the compressed FPGA output can be decoded successfully.

```
./decompress_host output_3 decompressed_output
```

Show that reconstructed image files are generated successfully in the decompressed_output folder.

- The decompressor reads the .gsc output produced by the FPGA compressor.
- Successful reconstruction confirms that the FPGA output follows the expected compressed-file format.

- The result is expected to be lossy because DCT coefficient quantisation removes information.

Part 5: Measure Compression Loss and PSNR

Goal: Quantify how closely the decompressed images match the original input images.

- Run the compare_pixels code giving the original images folder and decompressed images folder as arguments.

```
./compare_pixels images decompressed_output
```

- **PSNR interpretation:** a higher PSNR generally indicates that the reconstructed image is closer to the original image. >30 PSNR good reconstruction quality despite the lossy compression.